

Virtual Humans

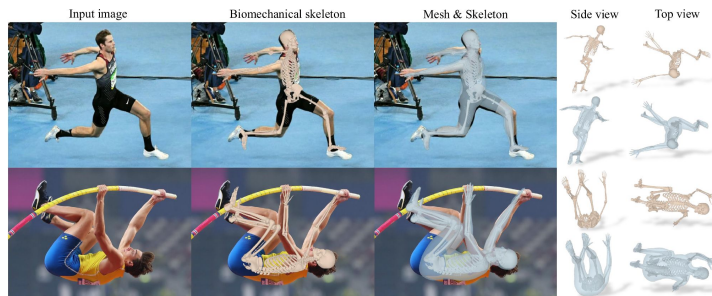
Motivation & Applications

- AR/VR
- Entertainment and Media
- Telepresence and Communication
- Education and Training
- Healthcare
- Robotics
- ...



Driving Signal

Animations



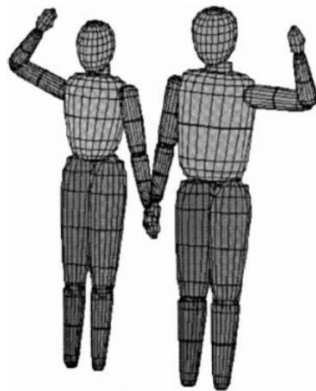
Why specialized representation?

- Human Bodies Are Articulated & Deformable
- Need for Semantic Control (like shape)
- Compactness & Efficiency
- Compatibility with Graphics Pipelines (like Blender)

Body models



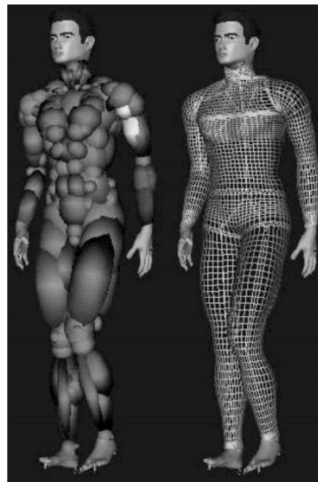
[Sminchisescu and Triggs '03]



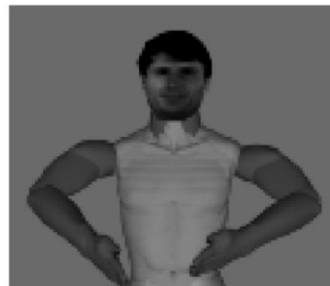
[Gavrilla, '96]



[Terzopoulos and Metaxas '93]



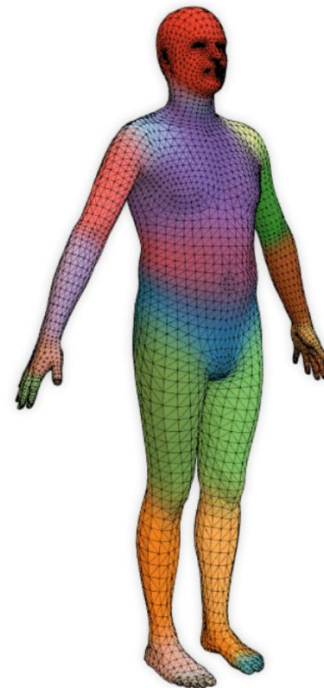
[Plänkers and Fua '01]



[Kakadiaris and Metaxas '00]

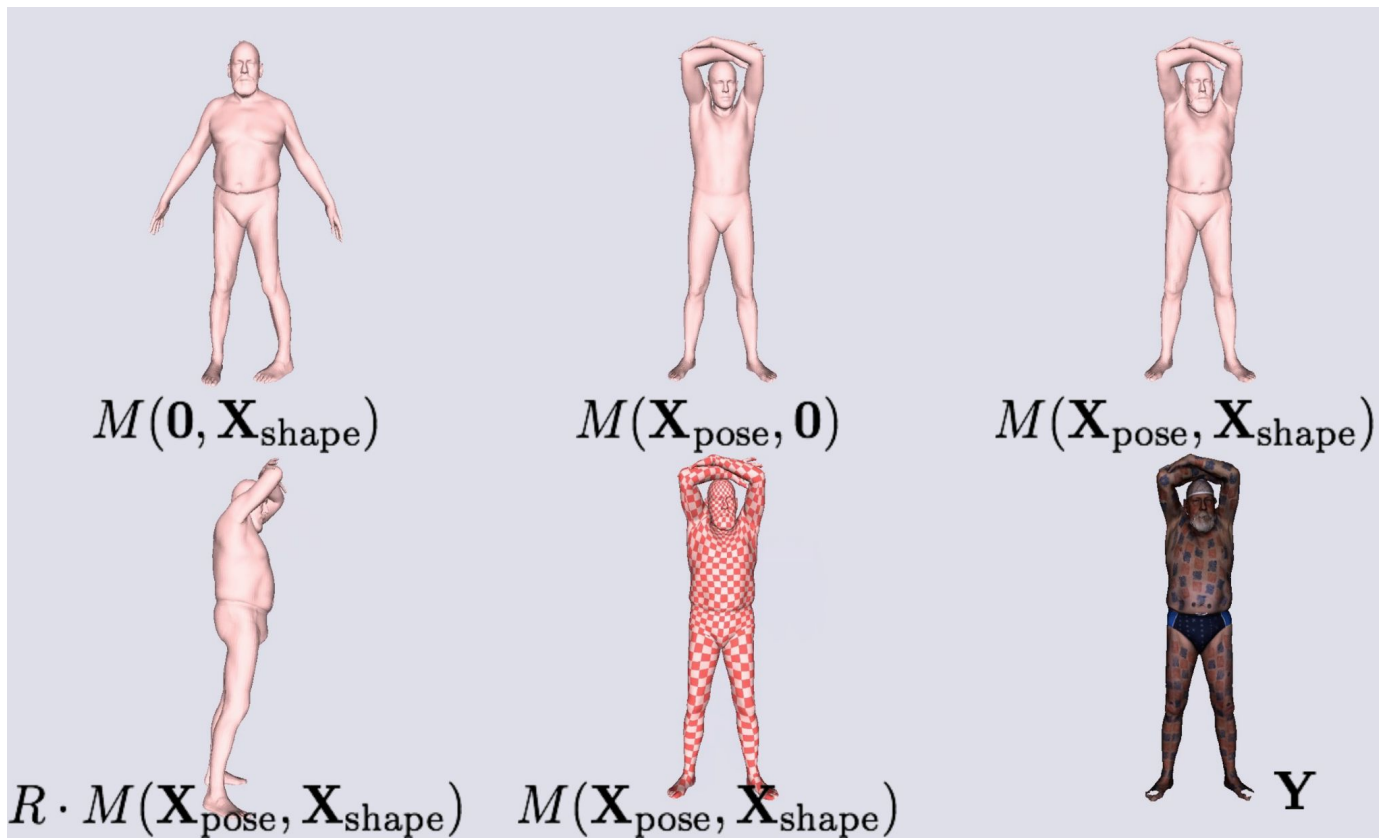


[Nevatia & Binford '73]



Loper et. al. (SMPL)

Body model is a function



Key idea: disentangle pose and shape

Pose: Change in 3D shape via articulation & local deformation



Intrinsic Shape: Individual shape variation



Why should the input X be the shape and pose?

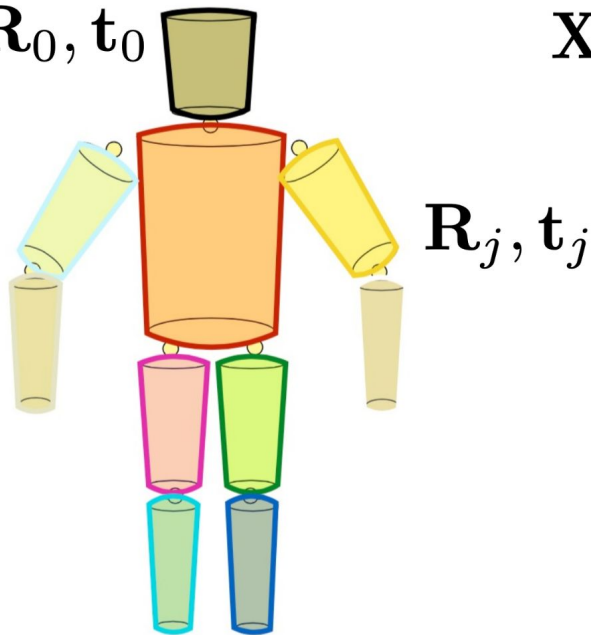
Notation: $\mathbf{X}_{\text{pose}} = \vec{\theta}$ $\mathbf{X}_{\text{shape}} = \vec{\beta}$

How do we parametrize the pose?

Parameterize every body part separately?

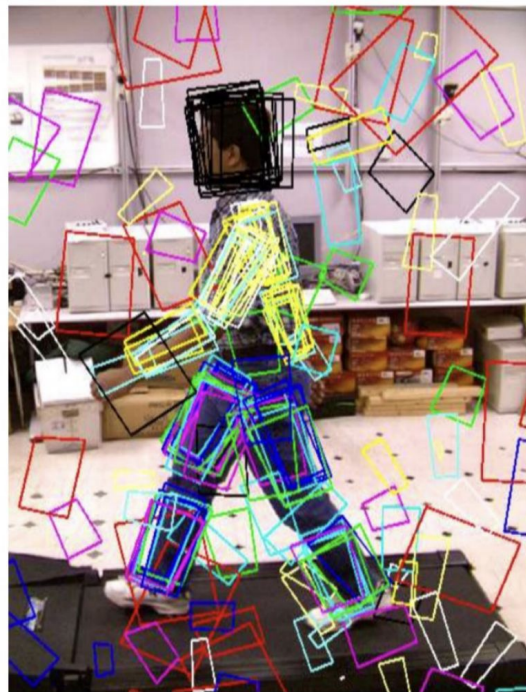
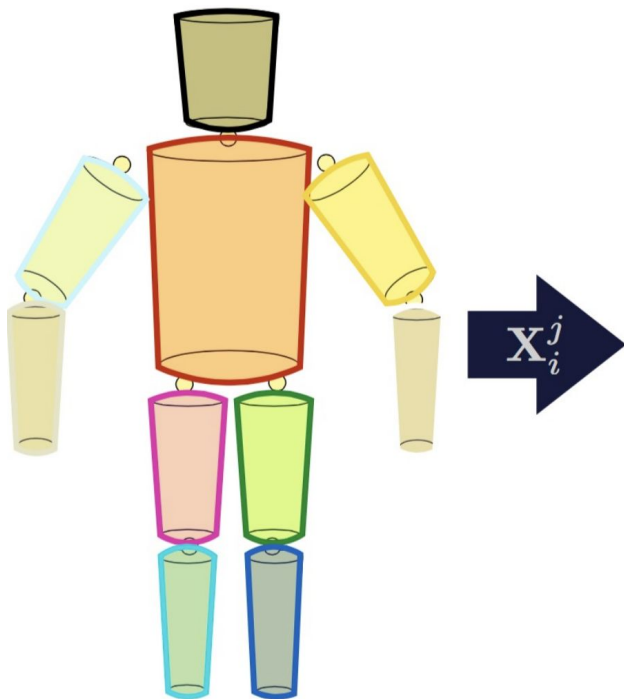
$\mathbf{R}_0, \mathbf{t}_0$

$$\mathbf{X}_{\text{pose}} = \{\mathbf{R}_0, \mathbf{t}_0, \dots, \mathbf{R}_N, \mathbf{t}_N\}$$



Problems ?

How do we parametrize the pose?

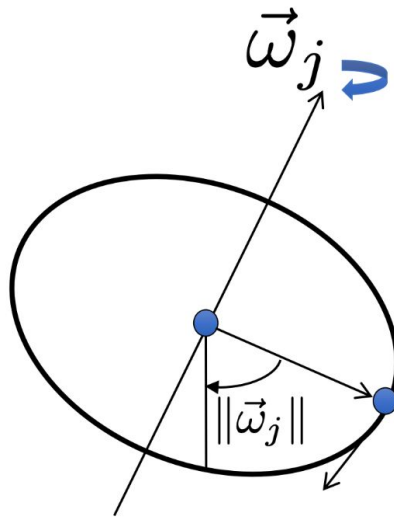


Articulation constraints are not satisfied!

Rotation with Exponential Maps

$\|\vec{\omega}_j\|$: Angle of rotation

$\vec{\omega}_j$: scaled axis of rotation



Rotation obtained with Rodrigues formula:

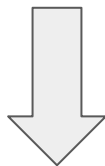
$$\text{Skew} \left(\begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix} \right) = \begin{bmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{bmatrix}$$

$$\mathbf{R} = e^{\hat{\vec{\omega}}} = \mathcal{I} + \hat{\vec{\omega}} \sin(\|\vec{\omega}_j\|) + \hat{\vec{\omega}}^2 (1 - \cos(\|\vec{\omega}_j\|))$$

Rotation with Exponential Maps

Say you are standing **on the ground** and you pick the direction of gravity to be the **negative z direction**.

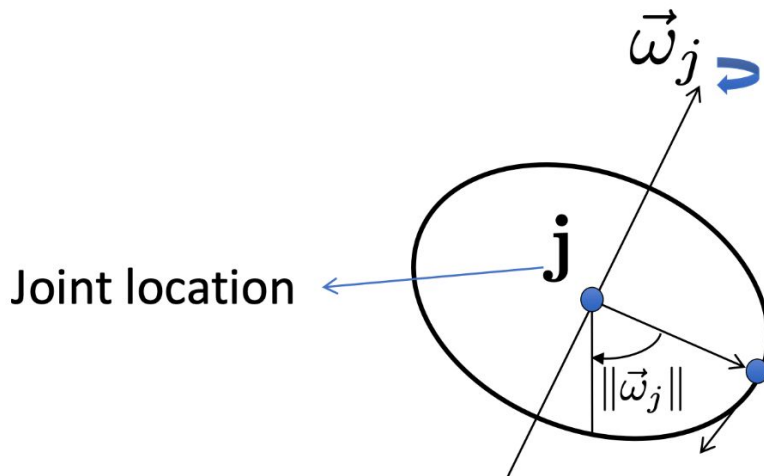
Turn to your **left**:
(axis , angle) = $\left(\begin{bmatrix} e_x \\ e_y \\ e_z \end{bmatrix}, \theta \right) = \left(\begin{bmatrix} 0 \\ 0 \\ -1 \end{bmatrix}, \frac{-\pi}{2} \right)$



$$\begin{bmatrix} 0 \\ 0 \\ \frac{\pi}{2} \end{bmatrix}$$

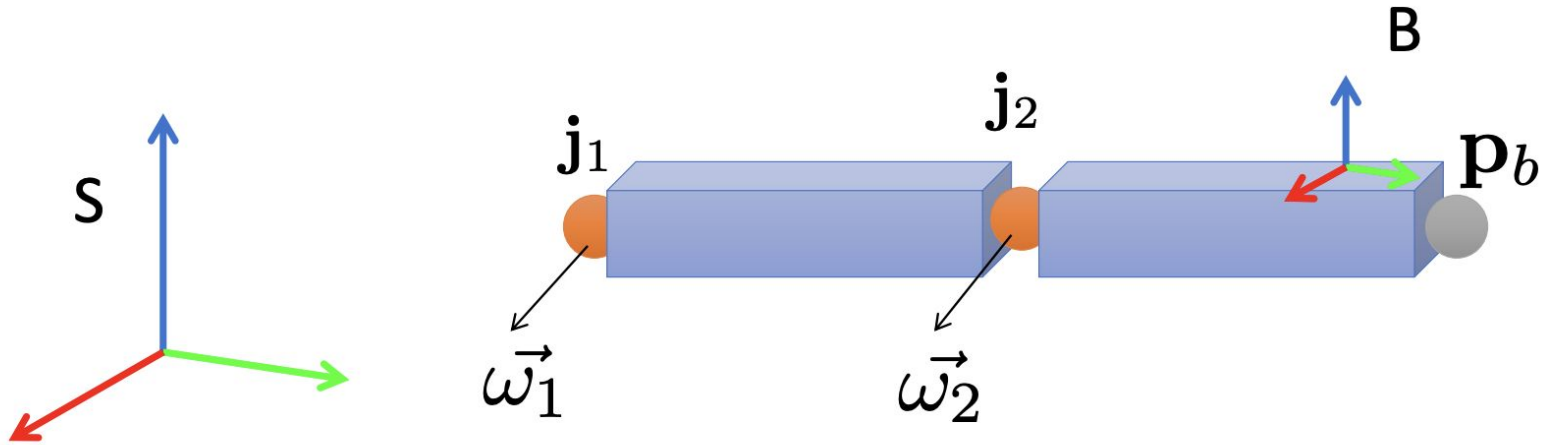
Joint Rigid Body Motion

The transformation associated with a rotational joint is:

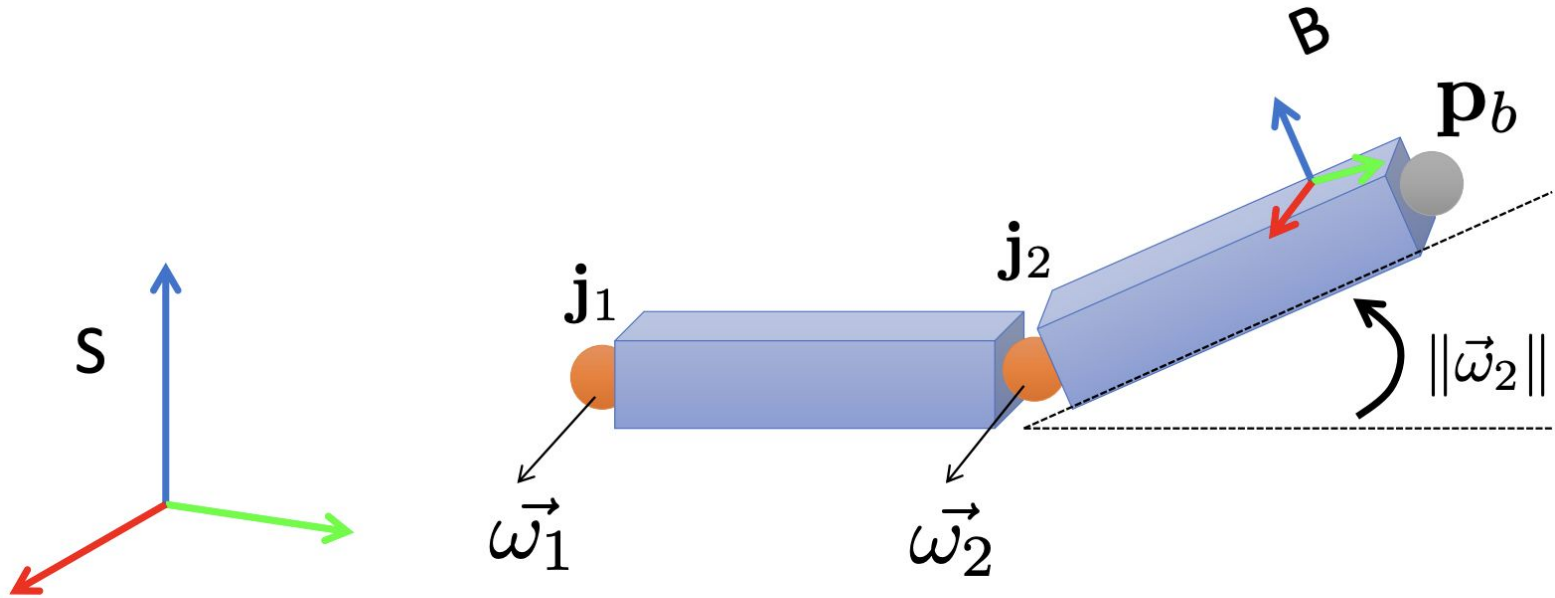


$$G(\vec{\omega}) = \begin{bmatrix} [e^{\vec{\omega}}]_{3 \times 3} & \mathbf{j}_{3 \times 1} - \mathbf{j}_{\text{parent}} \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix} \longrightarrow \text{Rigid Body Motion}$$

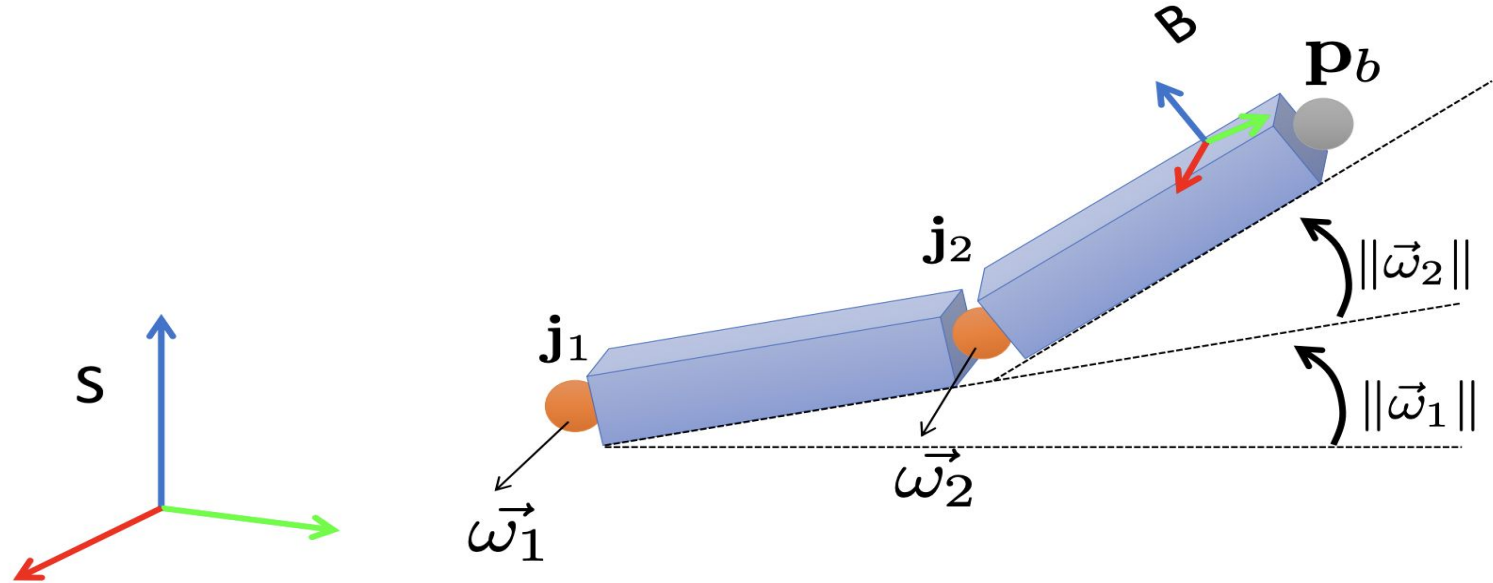
Kinematic Chains



Kinematic Chains



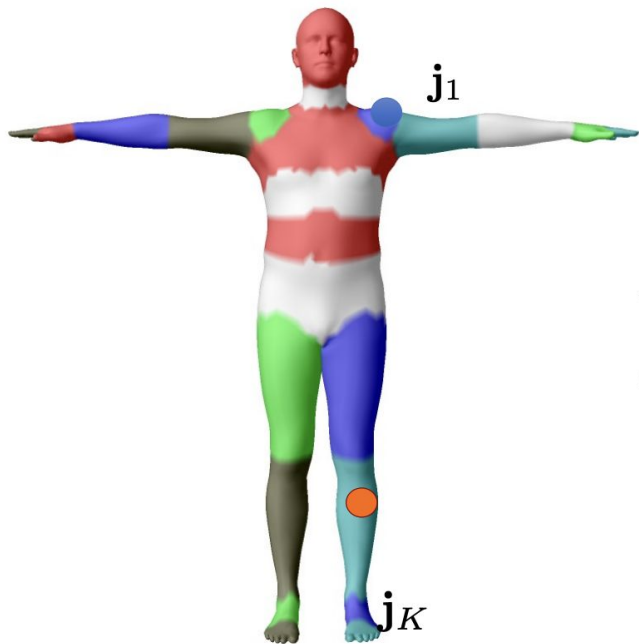
Kinematic Chains



The coordinates of the point in the spatial frame are:

$$\bar{\mathbf{p}}_s = G(\vec{\omega}_1, \vec{\omega}_2, \mathbf{j}_1, \mathbf{j}_2) = G(\vec{\omega}_1, \mathbf{j}_1) \boxed{G(\vec{\omega}_2, \mathbf{j}_2)} \bar{\mathbf{p}}_b$$

Pose parameters



T

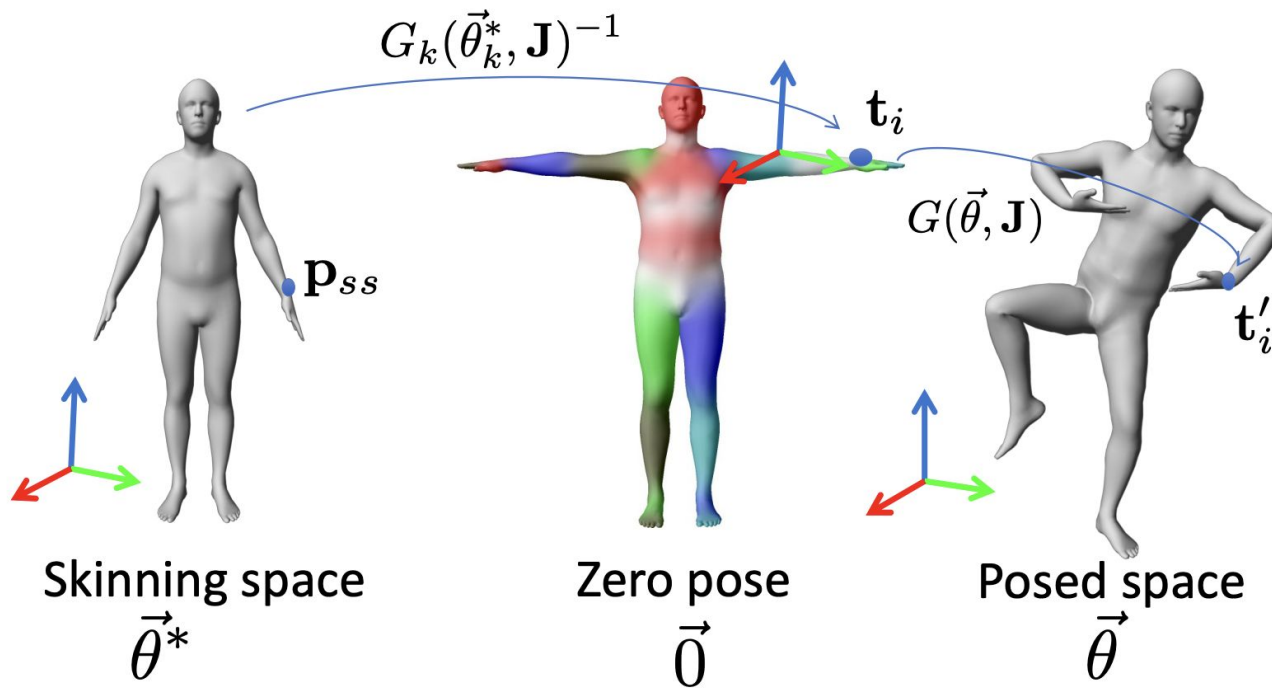
Given a set of joint locations

$$\mathbf{J} = (\underline{j_1}, \dots, \underline{j_K})^T$$

The pose defined as the vector of concatenated part axis-angles

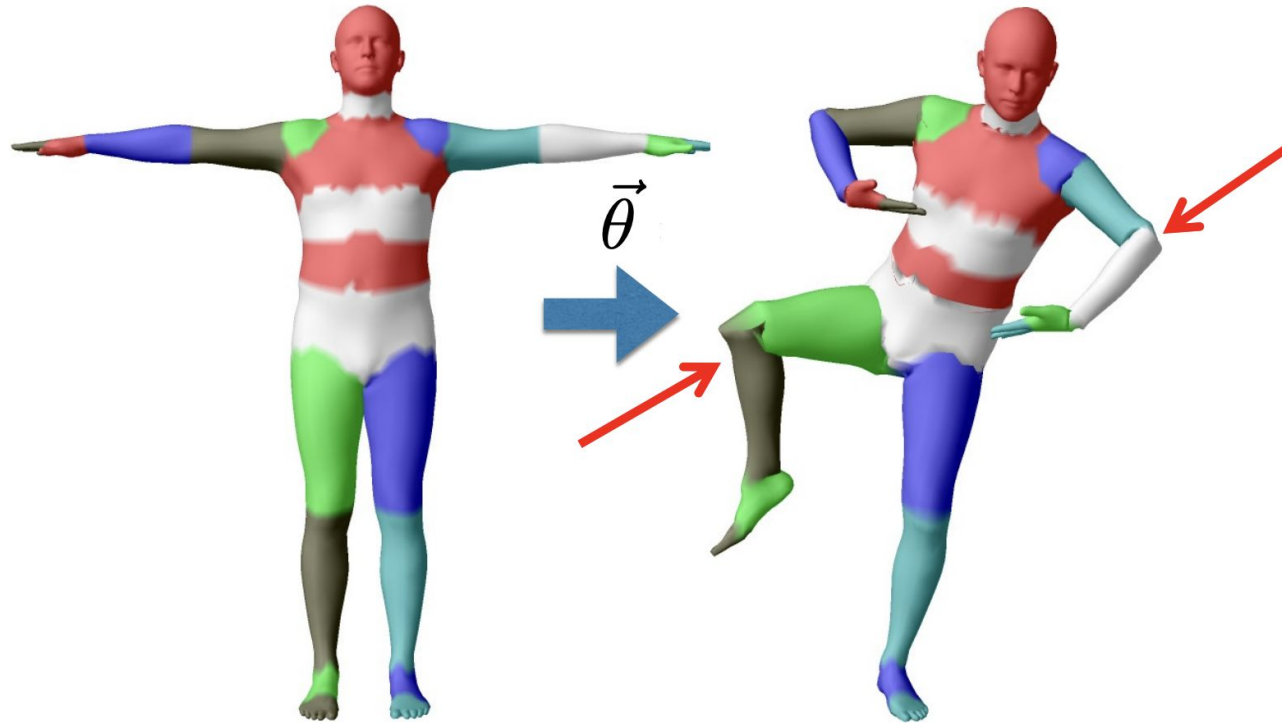
$$\vec{\theta} = (\underline{\vec{\omega}_1}, \dots, \underline{\vec{\omega}_k})^T$$

Binding Matrices



$$\boxed{\bar{\mathbf{t}}_i = G_k(\vec{\theta}_k^*, \mathbf{J})^{-1} \mathbf{p}_{ss}} \rightarrow G'_k(\vec{\theta}, \mathbf{J}) = G_k(\vec{\theta}, \mathbf{J}) \boxed{G_k(\vec{\theta}^*, \mathbf{J})^{-1}}$$

Kinematic Chain Problems

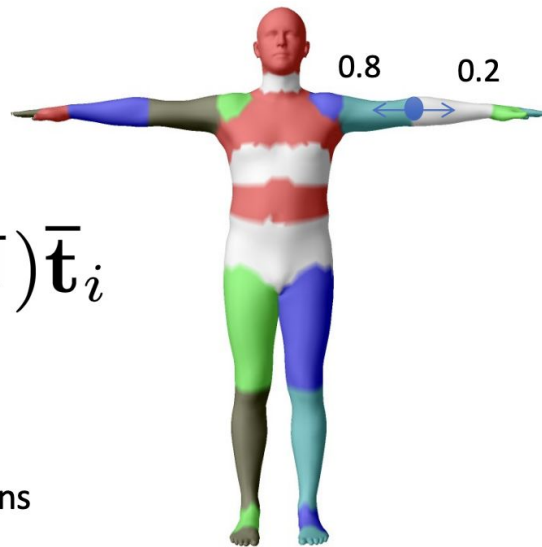


Linear Blend Skinning

$$\bar{\mathbf{t}}'_i = \sum_{k=1}^K w_{k,i} G'_k(\vec{\theta}, \mathbf{J}) \bar{\mathbf{t}}_i$$

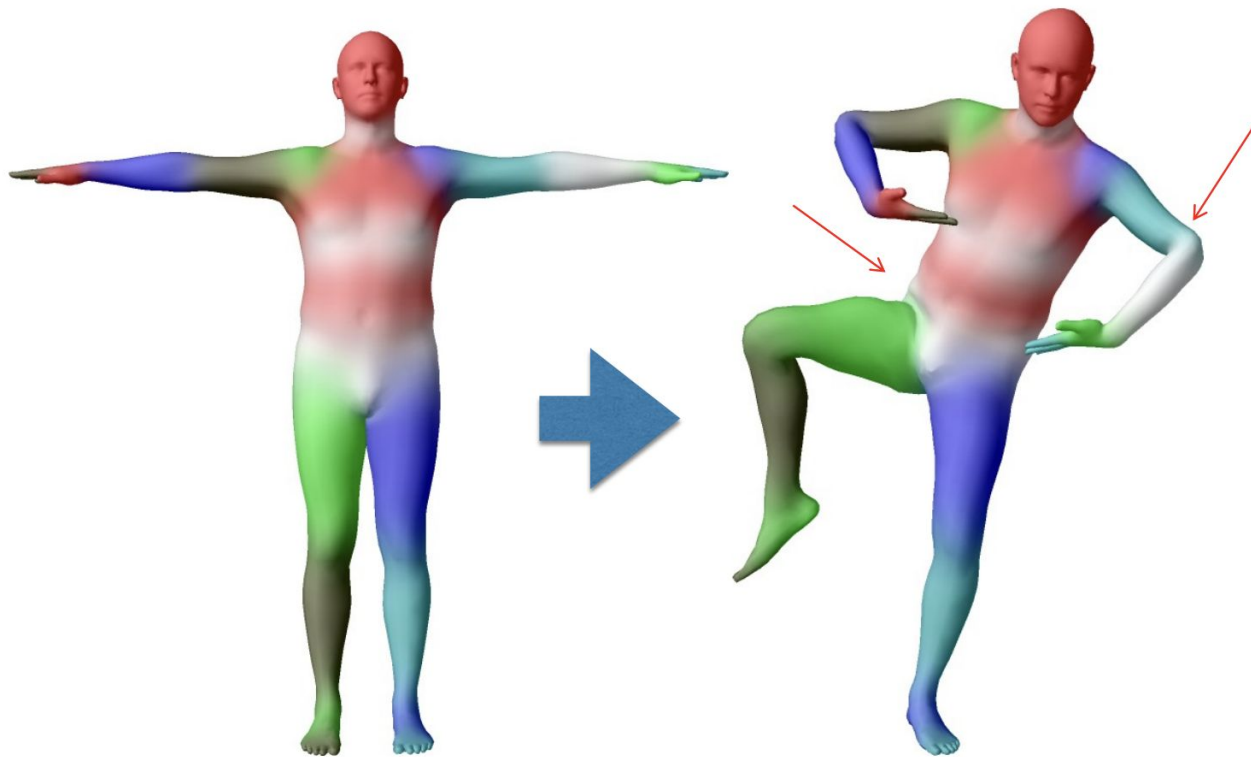
Blend weights

Part transformations



Points transformed as blended linear combination of joint transformation matrices

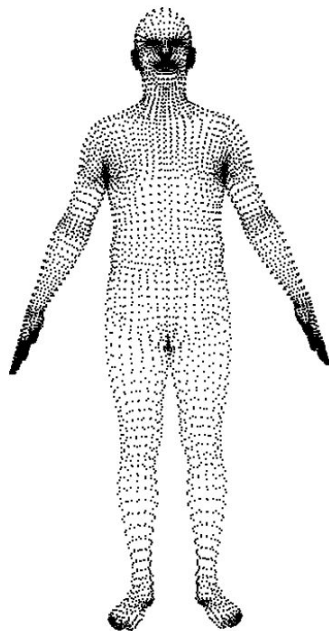
Linear Blend Skinning



Standard Skinning

Standard skinning produces vertices from...

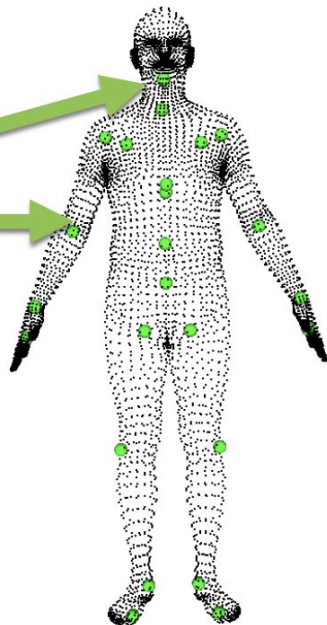
- Rest pose vertices: $\mathbf{T} \in \mathbb{R}^{3N}$
- Joint locations: $\mathbf{J} \in \mathbb{R}^{3K}$
- Weights: $\mathcal{W} \in \mathbb{R}^{N \times K}$
- Pose parameters: $\vec{\theta} \in \mathbb{R}^{3K}$



Standard Skinning

Standard skinning produces vertices from...

- Rest pose vertices: $\mathbf{T} \in \mathbb{R}^{3N}$
- Joint locations: $\mathbf{J} \in \mathbb{R}^{3K}$
- Weights: $\mathcal{W} \in \mathbb{R}^{N \times K}$
- Pose parameters: $\vec{\theta} \in \mathbb{R}^{3K}$



Standard Skinning

Standard skinning produces vertices from...

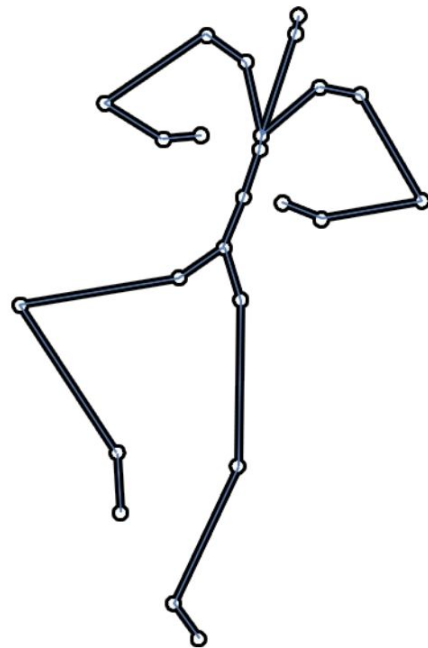
- Rest pose vertices: $\mathbf{T} \in \mathbb{R}^{3N}$
- Joint locations: $\mathbf{J} \in \mathbb{R}^{3K}$
- Weights: $\mathcal{W} \in \mathbb{R}^{N \times K}$
- Pose parameters: $\vec{\theta} \in \mathbb{R}^{3K}$



Standard Skinning

Standard skinning produces vertices from...

- Rest pose vertices: $\mathbf{T} \in \mathbb{R}^{3N}$
- Joint locations: $\mathbf{J} \in \mathbb{R}^{3K}$
- Weights: $\mathcal{W} \in \mathbb{R}^{N \times K}$
- Pose parameters: $\vec{\theta} \in \mathbb{R}^{3K}$

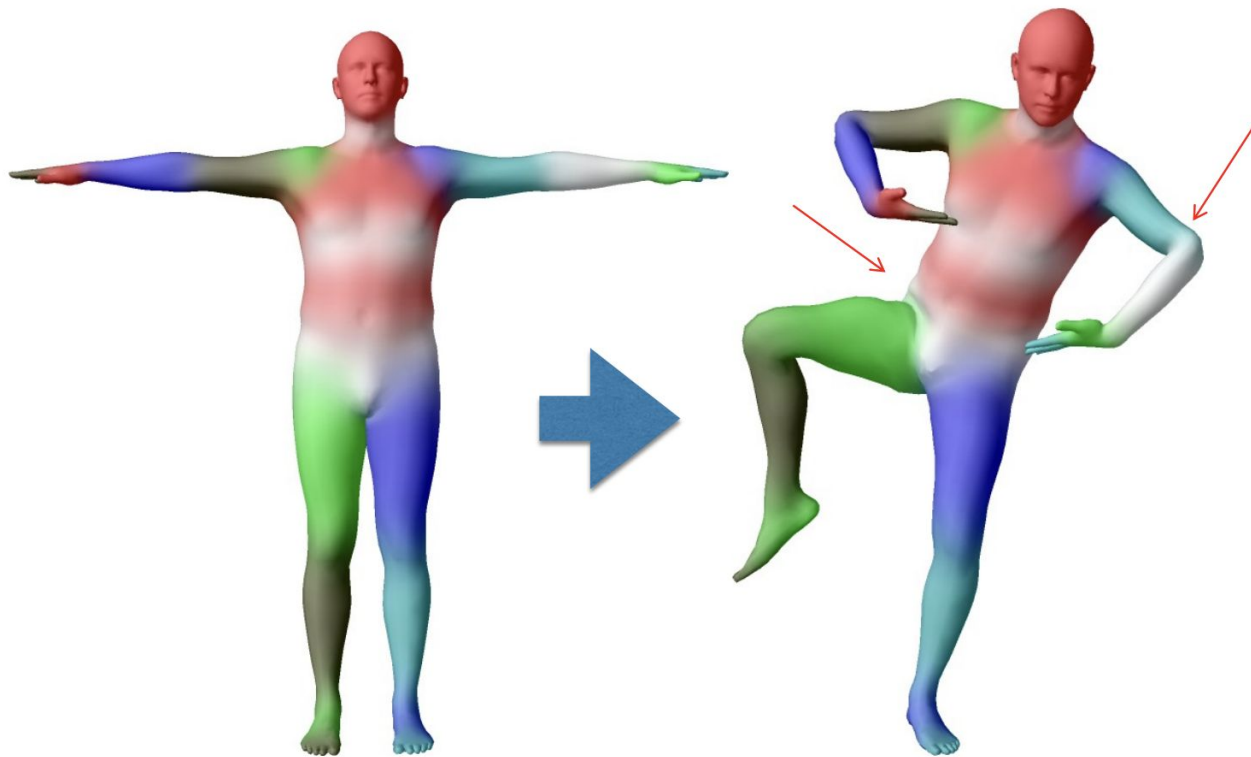


Standard Skinning

- Rest pose vertices: $\mathbf{T} \in \mathbb{R}^{3N}$
- Joint locations: $\mathbf{J} \in \mathbb{R}^{3K}$
- Weights: $\mathcal{W} \in \mathbb{R}^{N \times K}$
- Pose parameters: $\vec{\theta} \in \mathbb{R}^{3K}$

$$W(\mathbf{T}, \mathbf{J}, \mathcal{W}, \vec{\theta}) \mapsto \text{vertices}$$

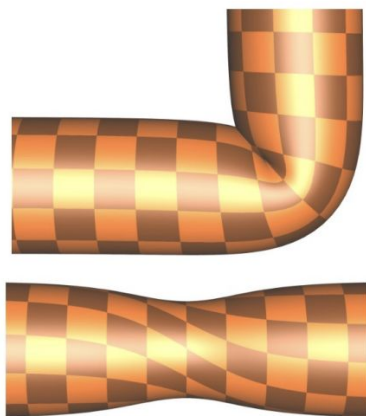
LBS problems



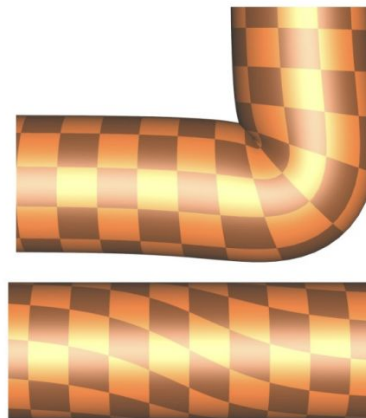
Standard Skinning

Problem: we want better pose-driven changes

LBS Skinning: collapse



DQ Skinning: bulge



[Kavan et al., 2012]

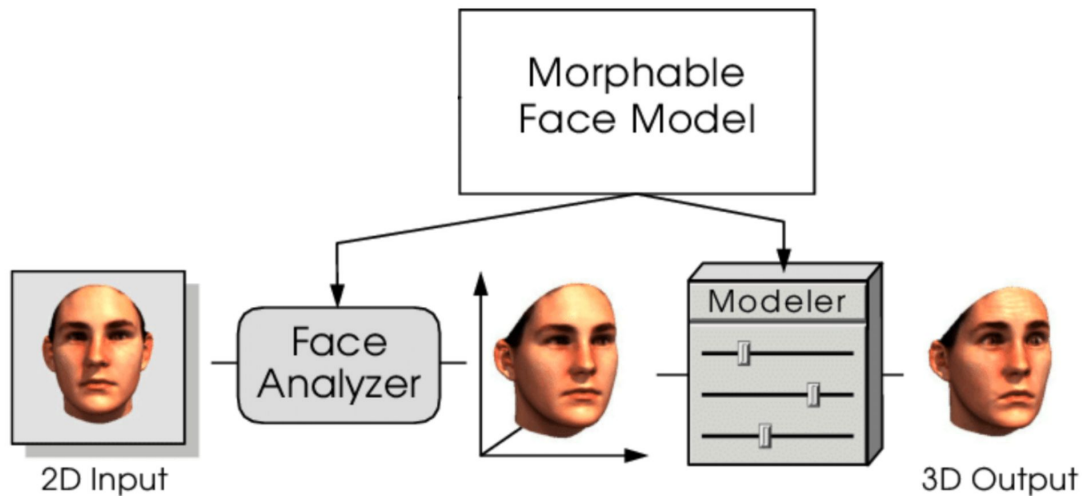
Blend Shapes

A blend shape is a set of vertex displacements in a rest pose



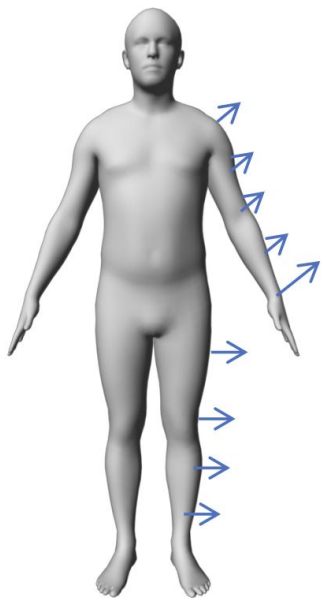
Blend Shapes

A blend shape is a set of vertex displacements in a rest pose



Blend Shapes

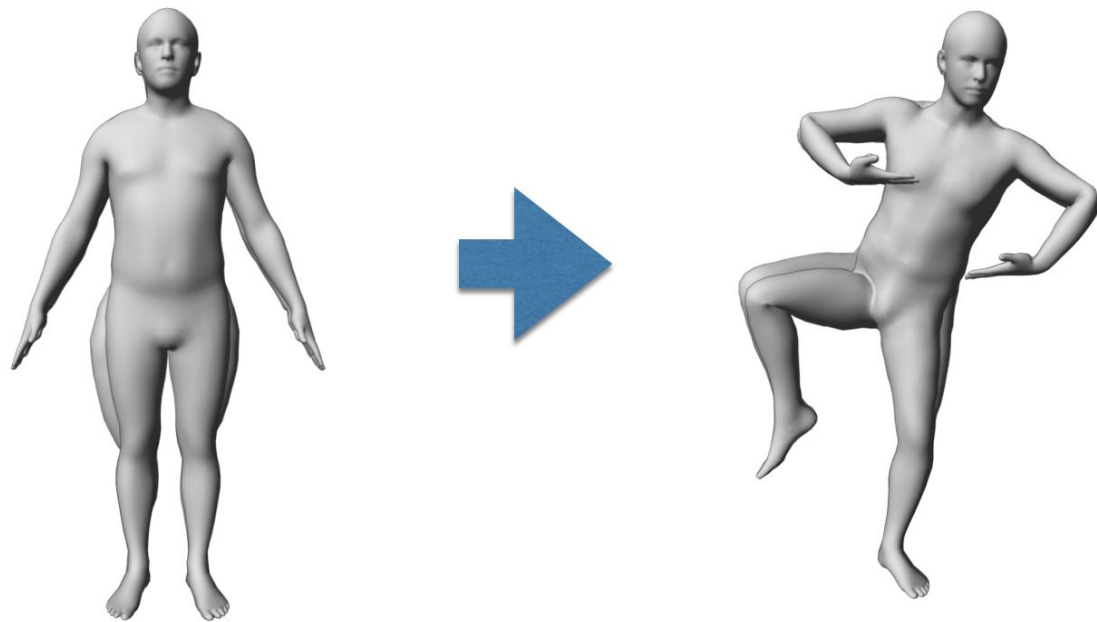
A blend shape is a set of vertex displacements in a rest pose



$$\mathbf{P} = \text{vec}\left(\begin{bmatrix} \Delta x_1 & \Delta y_1 & \Delta z_1 \\ & \vdots & \\ & \vdots & \\ \Delta x_N & \Delta y_N & \Delta z_N \end{bmatrix}\right) \xrightarrow{\text{Offset}} \mathbb{R}^{3N}$$

Pose Blend shapes

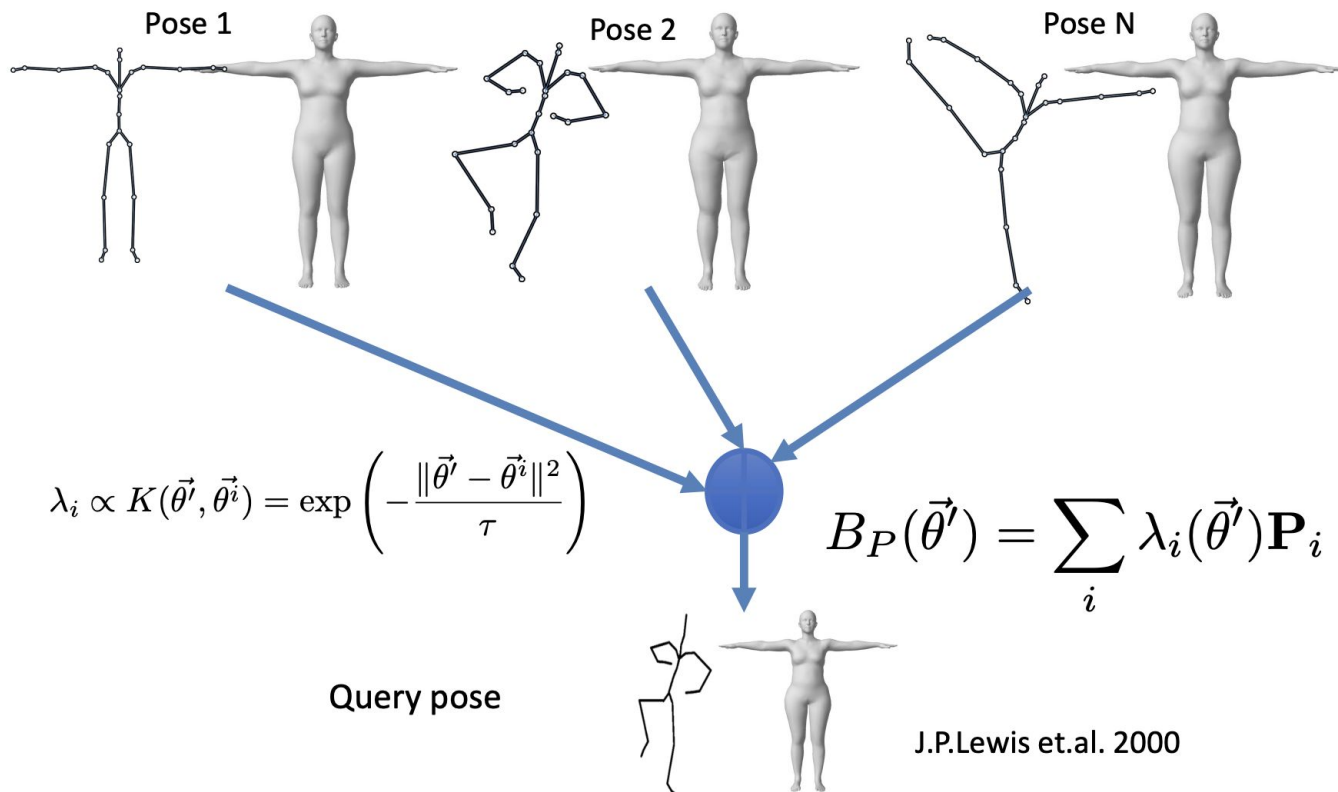
With blend shape correction



How to predict Blend Shapes ?

- Animators sculpt it manually!
- Time consuming, does not scale

Scattered Data Interpolation



Problems Scattered Data Interpolation

- Computationally expensive (need to find closest poses in a database)
- Does not extrapolate very well to novel poses

Problems

- If we don't use scattered data interpolation, how do we define pose blend shapes ?
- How do we model shape identity variations ?
- How to set the skinning parameters?

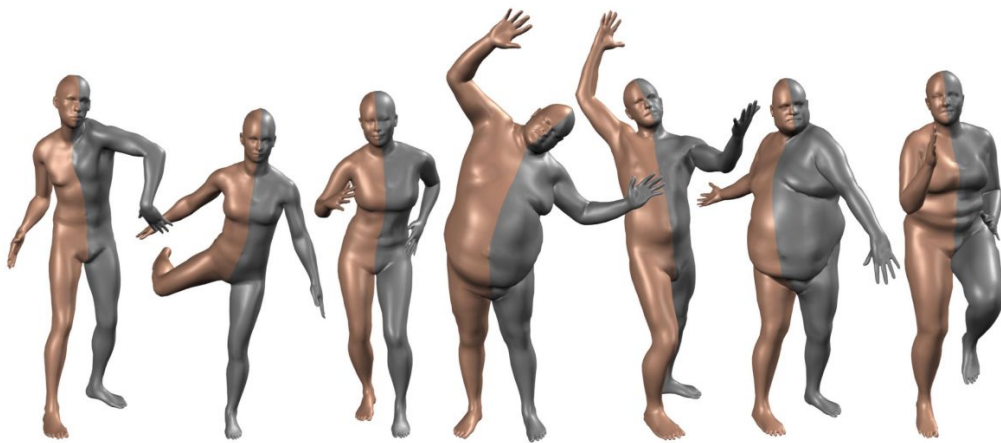
$$\mathbf{T} \in \mathbb{R}^{3N} \quad \mathbf{J} \in \mathbb{R}^{3K} \quad \mathcal{W} \in \mathbb{R}^{N \times K}$$

SMPL: A Skinned Multi-Person Linear Model

Matthew Loper^{*1,2} Naureen Mahmood^{†1} Javier Romero^{†1} Gerard Pons-Moll^{†1} Michael J. Black^{†1}

¹Max Planck Institute for Intelligent Systems, Tübingen, Germany

²Industrial Light and Magic, San Francisco, CA



Pipeline



(a) $\bar{\mathbf{T}}, \mathcal{W}$

Figure 3: SMPL model. (a) Template mesh with blend weights indicated by color and joints shown in white. (b) With identity-driven blendshape contribution only; vertex and joint locations are linear in shape vector $\vec{\beta}$. (c) With the addition of pose blend shapes in preparation for the split pose; note the expansion of the hips. (d) Deformed vertices reposed by dual quaternion skinning for the split pose.

Pipeline

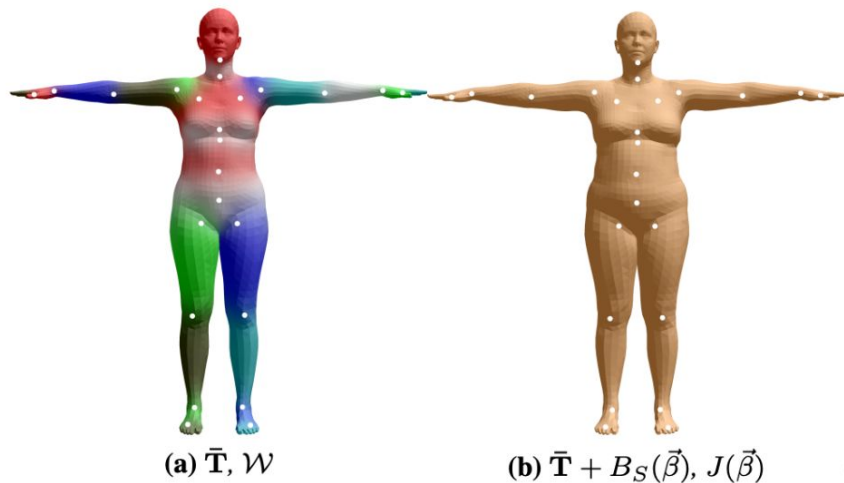


Figure 3: SMPL model. (a) Template mesh with blend weights indicated by color and joints shown in white. (b) With identity-driven blendshape contribution only; vertex and joint locations are linear in shape vector $\vec{\beta}$. (c) With the addition of of pose blend shapes in preparation for the split pose; note the expansion of the hips. (d) Deformed vertices reposed by dual quaternion skinning for the split pose.

Pipeline

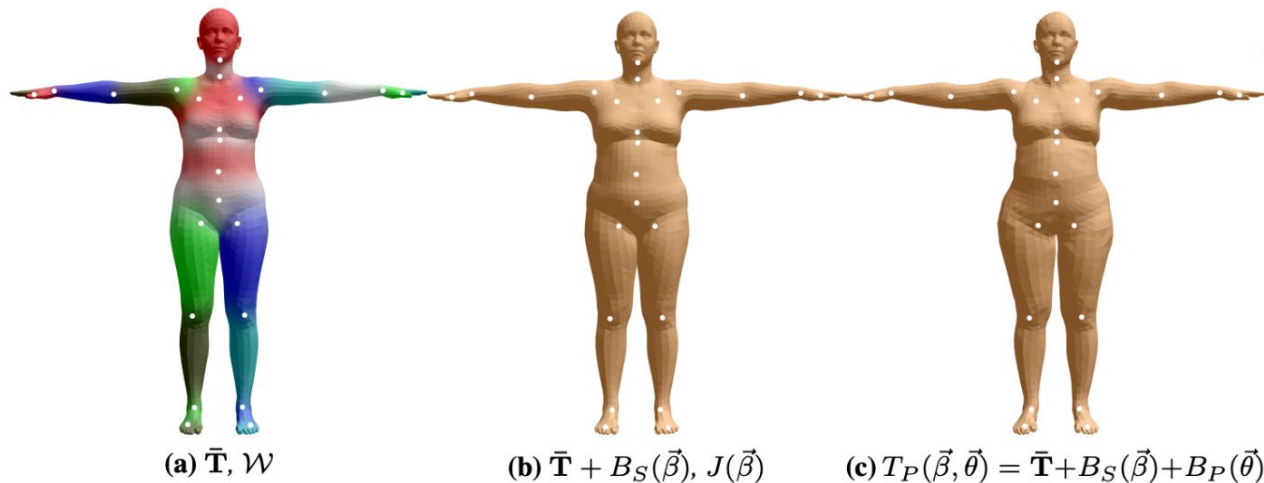


Figure 3: SMPL model. (a) Template mesh with blend weights indicated by color and joints shown in white. (b) With identity-driven blendshape contribution only; vertex and joint locations are linear in shape vector $\vec{\beta}$. (c) With the addition of pose blend shapes in preparation for the split pose; note the expansion of the hips. (d) Deformed vertices reposed by dual quaternion skinning for the split pose.

Pipeline

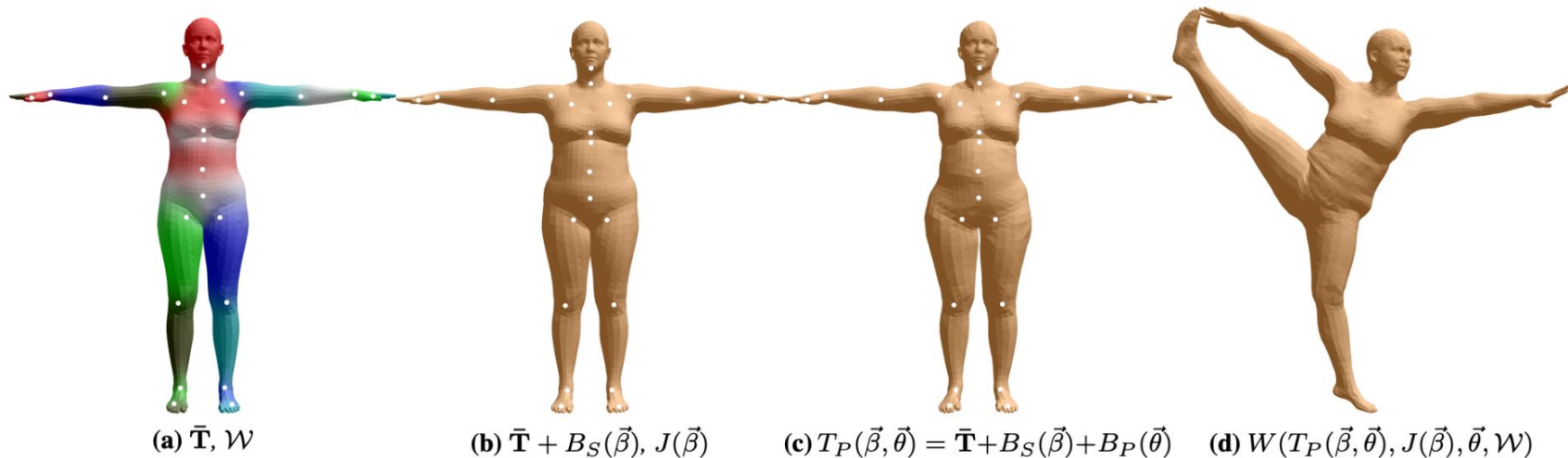


Figure 3: SMPL model. (a) Template mesh with blend weights indicated by color and joints shown in white. (b) With identity-driven blendshape contribution only; vertex and joint locations are linear in shape vector $\vec{\beta}$. (c) With the addition of of pose blend shapes in preparation for the split pose; note the expansion of the hips. (d) Deformed vertices reposed by dual quaternion skinning for the split pose.

Parameterized Skinning

Standard skinning $W(\mathbf{T}, \mathbf{J}, \mathcal{W}, \vec{\theta}) \mapsto \text{vertices}$

SMPL model

$M(\vec{\theta}, \vec{\beta}) = W(\mathbf{T}_F(\vec{\beta}, \theta), \mathbf{J}(\vec{\beta}), \mathcal{W}, \vec{\theta}) \mapsto \text{vertices}$

SMPL is skinning parameterized by pose $\vec{\theta}$
and shape $\vec{\beta}$

SMPL: BS are a *parametric function of pose*

$$W(\mathbf{T}, \mathbf{J}, \mathcal{W}, \vec{\theta})$$



$$W(T(\theta), \mathbf{J}, \mathcal{W}, \vec{\theta})$$

Parameterized Skinning

$$W(T(\theta), \mathbf{J}, \mathcal{W}, \vec{\theta}) \mapsto \text{vertices}$$

$$T(\vec{\theta}) = \mathbf{T} + B_P(\vec{\theta})$$

$$B_P(\vec{\theta}) = \sum_i^{|f(\vec{\theta})|} f_i(\vec{\theta}) \mathbf{P}_i$$

Each is
a blend shape

Parameterized Skinning

- What function $f(\vec{\theta})$?

$$B_P(\vec{\theta}) = \sum_i^{|f(\vec{\theta})|} f_i(\vec{\theta}) \mathbf{P}_i$$

- Simplest possible:

$$f(\vec{\theta}) = \vec{\theta}$$

Unconstrained parametrization

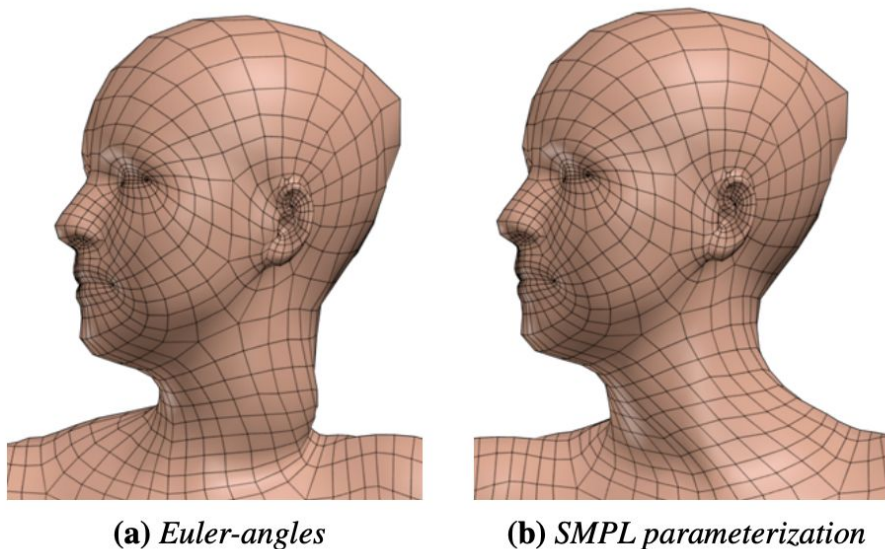


Figure 16: Parameterizing pose blend shapes. (a) *Pose blend shapes parameterized by Euler angles cause significant problems.* (b) *our proposed parameterization allows the head to rotate in either direction with natural deformations.*

Parameterized Skinning

- What function $f(\vec{\theta})$?

$$B_P(\vec{\theta}) = \sum_i \frac{|f(\vec{\theta})|}{|f(\vec{\theta})|} f_i(\vec{\theta}) \mathbf{P}_i$$

- Idea: we consider $f(\vec{\theta})$ as the vectorized joint rotation matrices
- Blend shapes are *linear in rotation matrix elements*

Parameterized Skinning

$$B_P(\vec{\theta}) = \sum_i^{|f(\vec{\theta})|} f_i(\vec{\theta}) \mathbf{P}_i$$

$$\vec{\theta} = (\vec{\omega}_1, \dots, \vec{\omega}_k)^T$$

$$e^{\hat{\omega}_1} - \mathcal{I} \quad e^{\hat{\omega}_K} - \mathcal{I}$$

$$f(\vec{\theta}) = \left[\underbrace{\bar{e}_{1,1}^{\hat{\omega}_1} \dots \bar{e}_{3,3}^{\hat{\omega}_1}} \quad \dots \quad \underbrace{\bar{e}_{1,1}^{\hat{\omega}_K} \dots \bar{e}_{3,3}^{\hat{\omega}_K}} \right]$$

9 elements of the rotation matrix-> We learn 9xK=207 blendshapes

Joint Location Estimation

- How to get the joints $\mathbf{J}$ for a new shape? What is the simplest way?
- Joints are considered linear in rest vertices (much like in Allen et al. '06)

$$\mathbf{J} = J(\mathbf{T}; \mathcal{J}) = \mathcal{J}\mathbf{T}$$



Joint regressor matrix

Joint Location Estimation

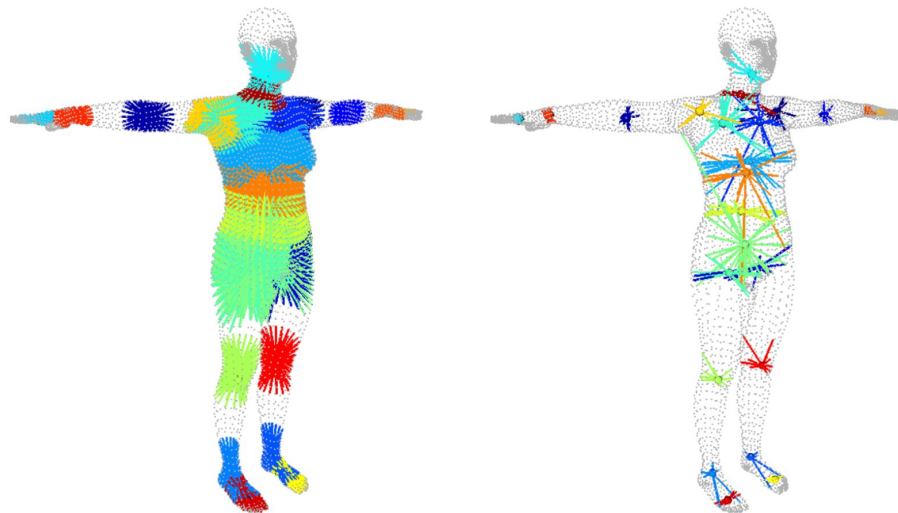


Figure 7: Joint regression. *(left) Initialization. Joint locations can be influenced by locations on the surface, indicated by the colored lines. We assume that these influences are somewhat local. (right) Optimized. After optimization we find a sparse set of vertices and associated weights influencing each joint.*

Adding a shape space

Problem: want a shape space with different identities

$$W(T(\vec{\theta}), J(\mathbf{T}), \mathcal{W}, \vec{\theta}) \mapsto \text{vertices}$$

$$T(\vec{\theta}) = \mathbf{T} + B_P(\vec{\theta})$$

$$\text{Pose contribution} \left\{ B_P(\vec{\theta}) = \sum_i^{|f(\vec{\theta})|} f_i(\vec{\theta}) \mathbf{P}_i \right.$$

Adding a shape space

Solution: add blend shapes linear with *beta*

$$W(T(\vec{\theta}, \vec{\beta}), J(\vec{\beta}), \mathcal{W}, \vec{\theta}) \mapsto \text{vertices}$$

$$T_P(\vec{\theta}, \vec{\beta}) = \mathbf{T} + B_P(\vec{\theta}) + B_S(\vec{\beta})$$

$$\text{Pose contribution} \left\{ B_P(\vec{\theta}) = \sum_i^{|f(\vec{\theta})|} f_i(\vec{\theta}) \mathbf{P}_i \right.$$

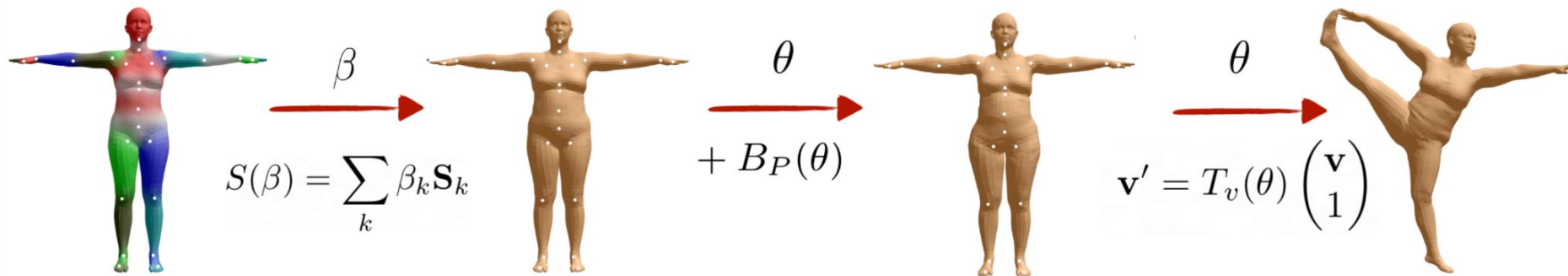
$$\text{Shape contribution} \left\{ B_S(\beta) = \sum_j^{|\beta|} \beta_j S_j \right.$$

SMPL

Additive Model

$$\bar{\mathbf{t}}'_i = \sum_{k=1}^K w_{k,i} G'_k(\vec{\theta}, J(\vec{\beta})) (\bar{\mathbf{t}}_i + \mathbf{b}_{S,i}(\vec{\beta}) + \mathbf{b}_{P,i}(\vec{\theta}))$$

Joint locations Vertices Shape bs Pose bs



What are the model parameters?

How to acquire these?

$\mathbf{S} \in \mathbb{R}^{|\beta| \times (3N)}$: shape bases $P \in \mathbb{R}^{9|\mathcal{H}| \times N}$: pose blend bases

$\mathbf{W} \in \mathbb{R}^{|\mathcal{H}| \times N}$: deformation weights

SMPL

$$M(\vec{\theta}, \vec{\beta}; \mathbf{T}, \mathcal{S}, \mathcal{P}, \mathcal{W}, \mathcal{J})$$

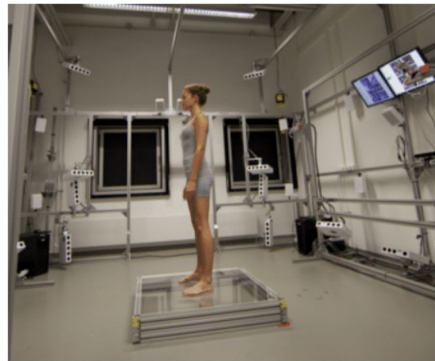
pose shape

Input Model parameters to be learned from data

$\mathbf{T}$	Template (average shape)	} $\mathbf{w}$
$\mathcal{S}$	Shape blend shape matrix	
$\mathcal{P}$	Pose blend shape matrix	
$\mathcal{W}$	Blendweights matrix	
$\mathcal{J}$	Joint regressor matrix	

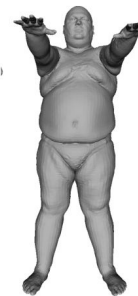
Training data

- Non-Rigidity
- Occlusion & Self-Occlusion
- Lighting and Reflectivity
- Calibration & Synchronization



Training model

$$\mathbf{w} = \arg \min_{\mathbf{w}} \sum_j \|M(\vec{\theta}, \vec{\beta}; \mathbf{w}) -$$



$\|^2$



Training model

Ideally one wants to find the model parameters that **minimize** a single objective measuring the distance between **model** and **registrations**

$$\arg \min_{\mathbf{T}, \mathcal{S}, \mathcal{P}, \mathcal{W}, \mathcal{J}} \sum_j \min_{\vec{\theta}_j, \vec{\beta}_j} \|M(\vec{\theta}_j, \vec{\beta}_j; \mathbf{T}, \mathcal{S}, \mathcal{P}, \mathcal{W}, \mathcal{J}) - \mathbf{V}_j\|^2$$


Model Registrations

Training details

- $\mathcal{P}, \mathcal{W}, \mathcal{J}$ are trained from our **multipose** dataset
- $\mathcal{P}$ regularized towards zero (ridge regression)
- $\mathcal{W}$ regularized towards initialization
- $\mathcal{J}$ regularized towards predicting part boundary centers and is forced to be sparse
- $\mathbf{T}, \mathcal{S}$ are trained from our **multishape** dataset

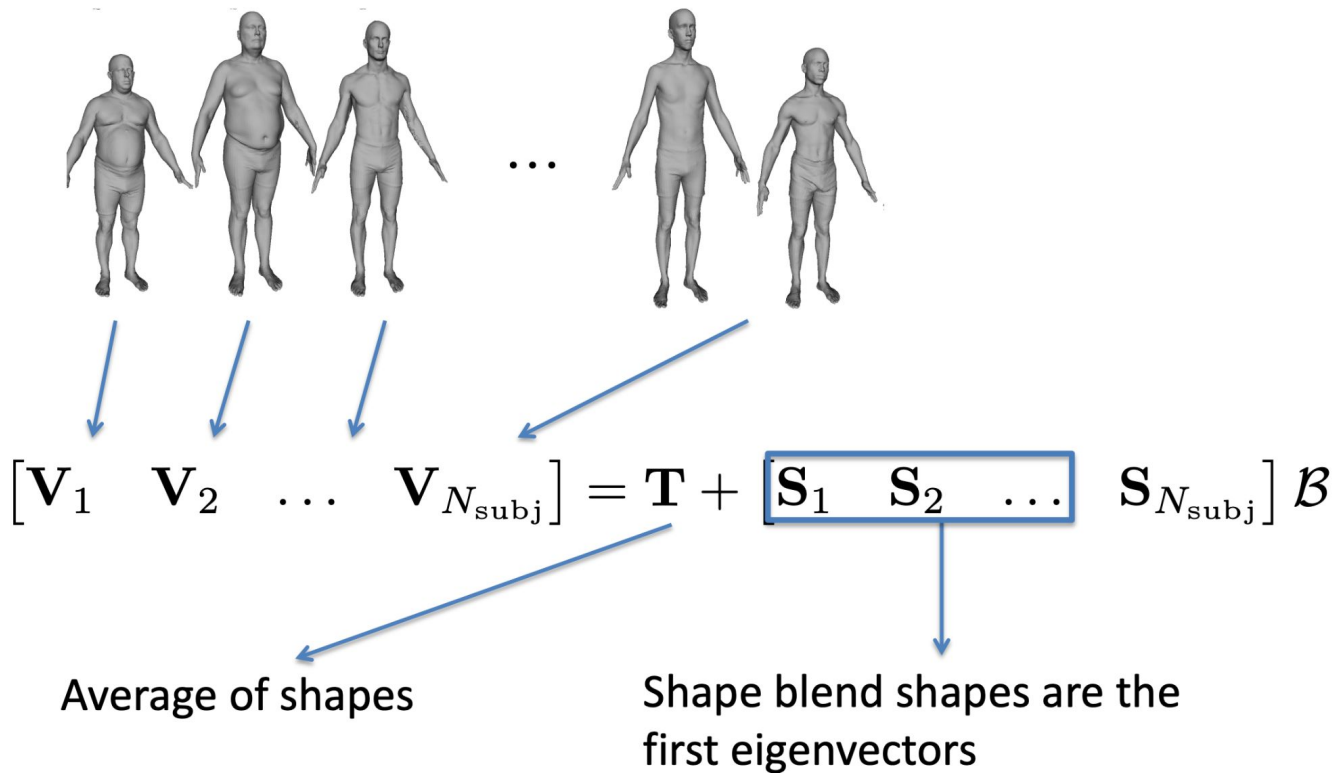
Number of Parameters Learned

For a model with 6890 vertices:

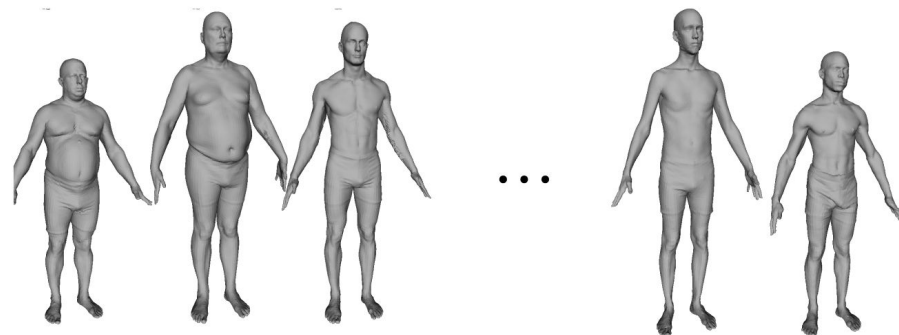
- $\mathcal{P}$ $9 \times 23 \times 6890 = 4,278,690$
- $\mathcal{W}$ $4 \times 3 \times 6890 = 82,680$
- $\mathcal{J}$ $3 \times 6890 \times 23 \times 3 = 1,426,230$
- $\mathbf{T}, \mathcal{S}$ $3 \times 6890 + 3 \times 6890 \times 10 \text{blendshapes} = 227,370$

A total of 6.014.970 parameters are learned

Shape blend shapes



Shape blend shapes



$$[\mathbf{V}_1 \quad \mathbf{V}_2 \quad \dots \quad \mathbf{V}_{N_{\text{subj}}}] \approx \mathbf{T} + \mathcal{SB}$$

Average of shapes

Shape blend shapes matrix

Shape blend shapes

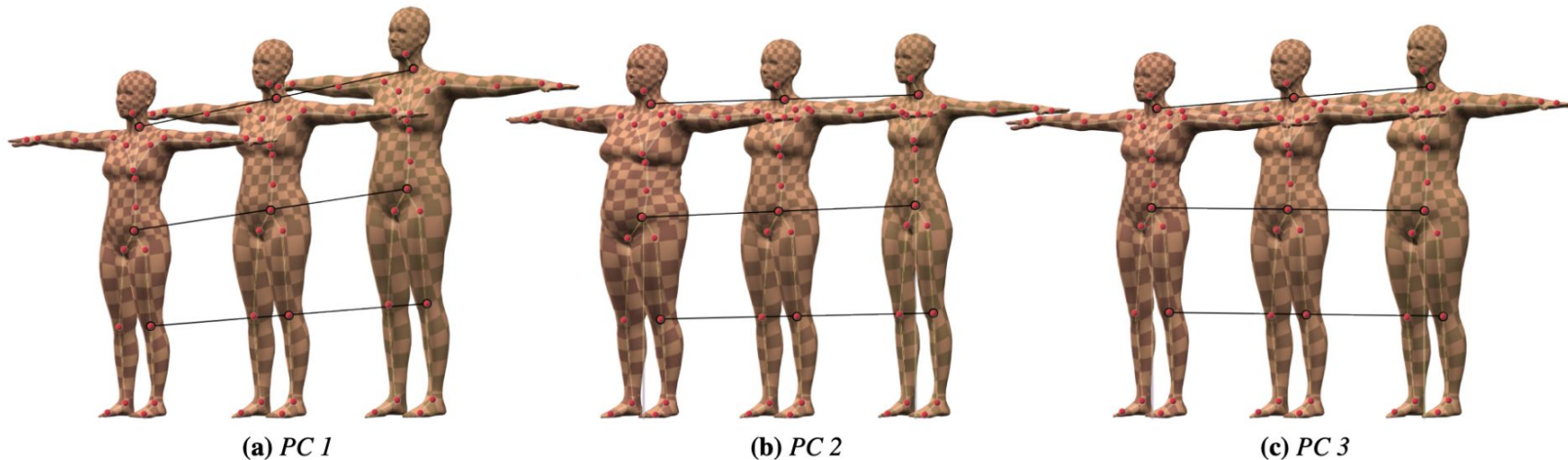
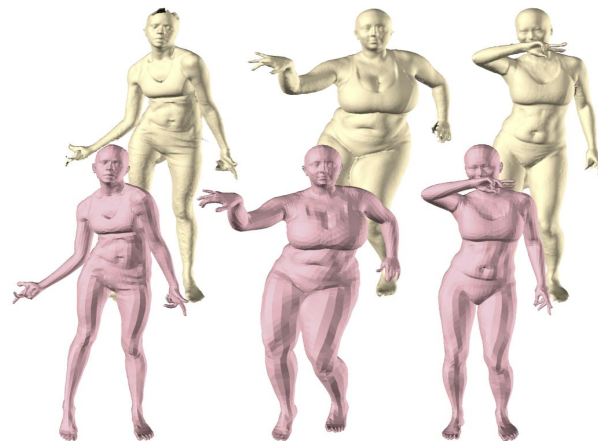
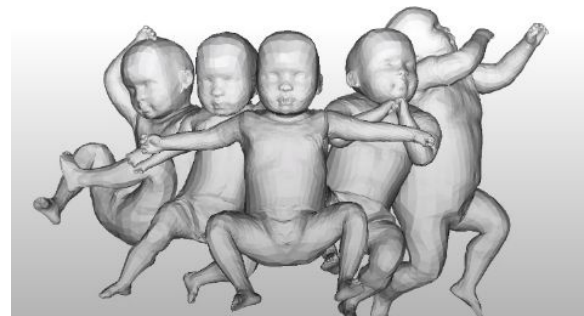
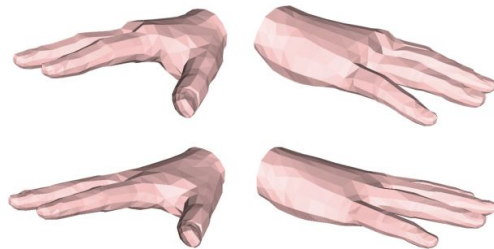
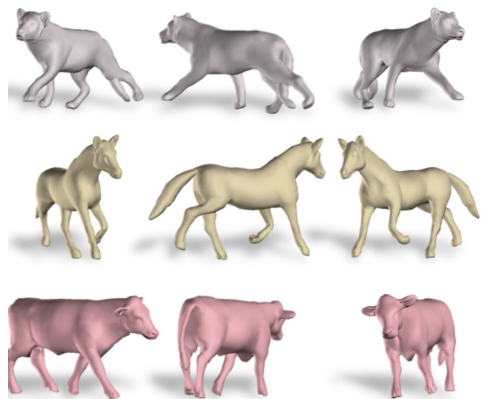


Figure 8: Shape blend shapes. The first three shape principal components of body shape are shown. PC1 and PC2 vary from -2 to +2 standard deviations from left to right, while PC3 varies from -5 to +5 standard deviations to make the shape variation more visible. Joint locations (red dots) vary as a function of body shape and are predicted using the learned regressor, $\mathcal{J}$.

SMPL successors



"See" the person in 3D



Why is it Hard?

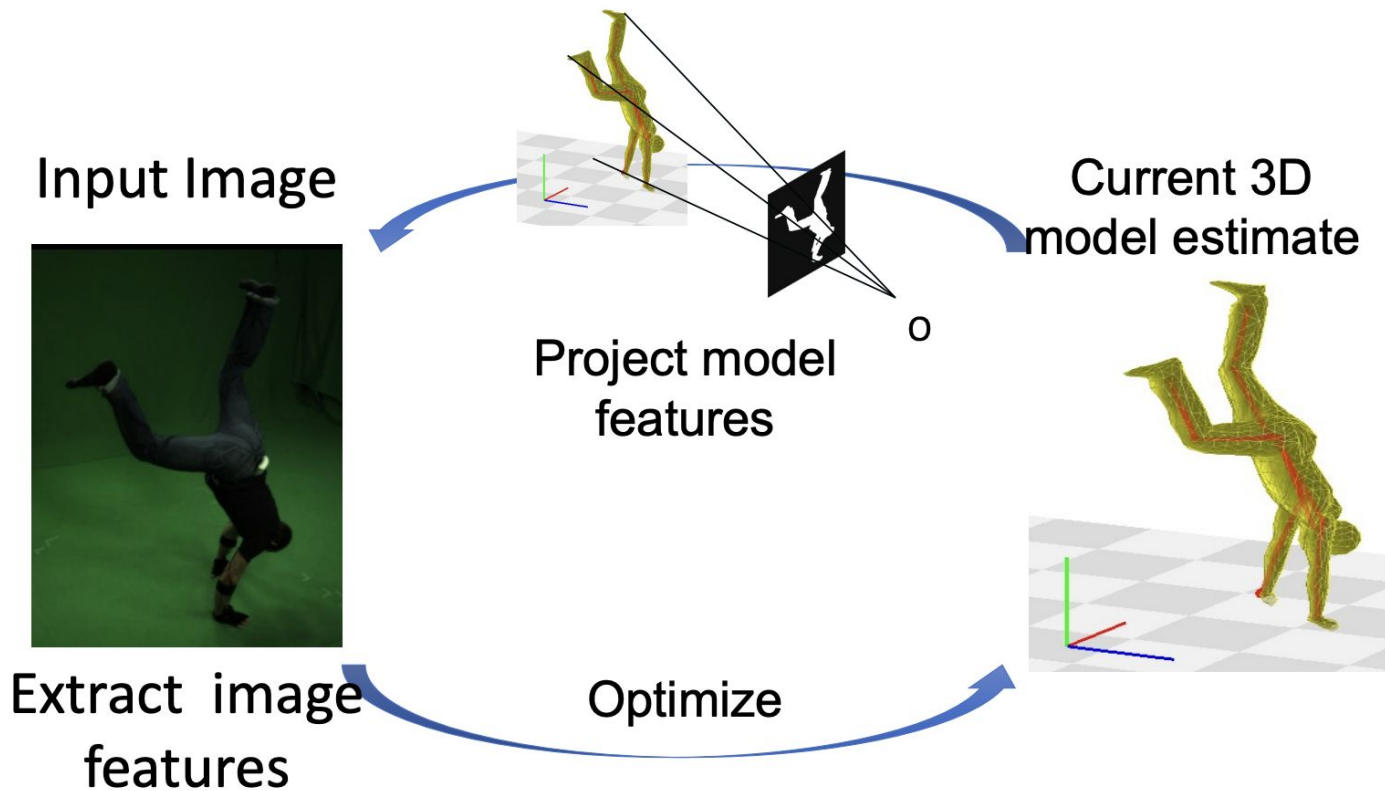
- Many degrees of freedom.
- Highly Dynamic / Skinning/ Clothing / Outdoor.
- Large variability and individuality of Motion patterns.



Problem formulation



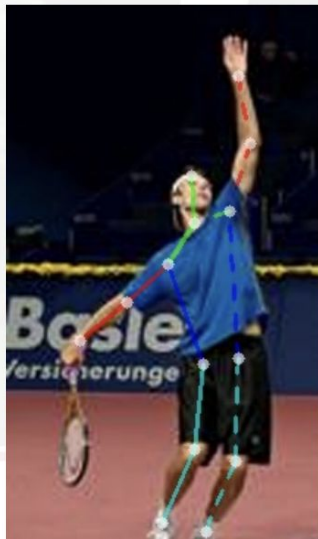
How to "match" a 3D Model with a 2D image?



Optimization based approach

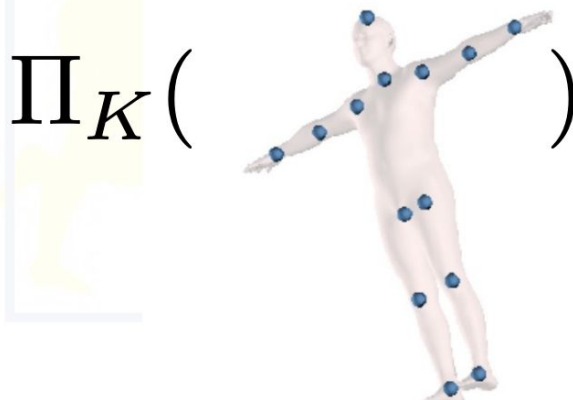
1. Extract features

2D joints are quite reliable



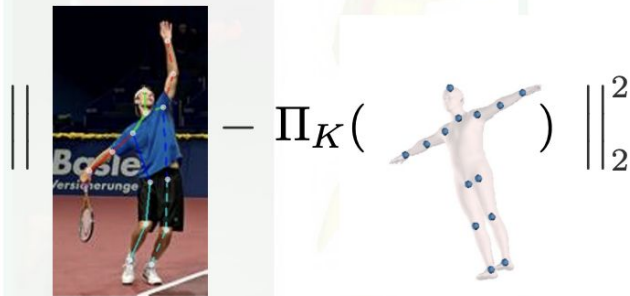
2. Predict and match

Match projection of 3D joints with 2D joints



3. Optimize

Min. L_2 dist. b/w image and projected joints



Problem: Depth Ambiguity

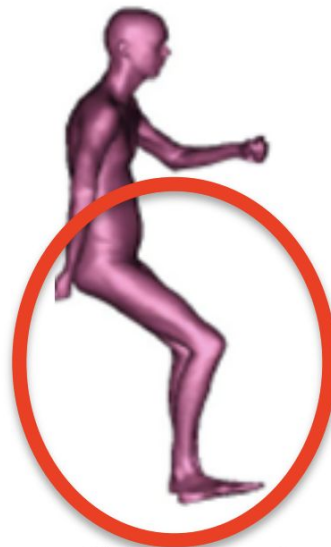


Input image



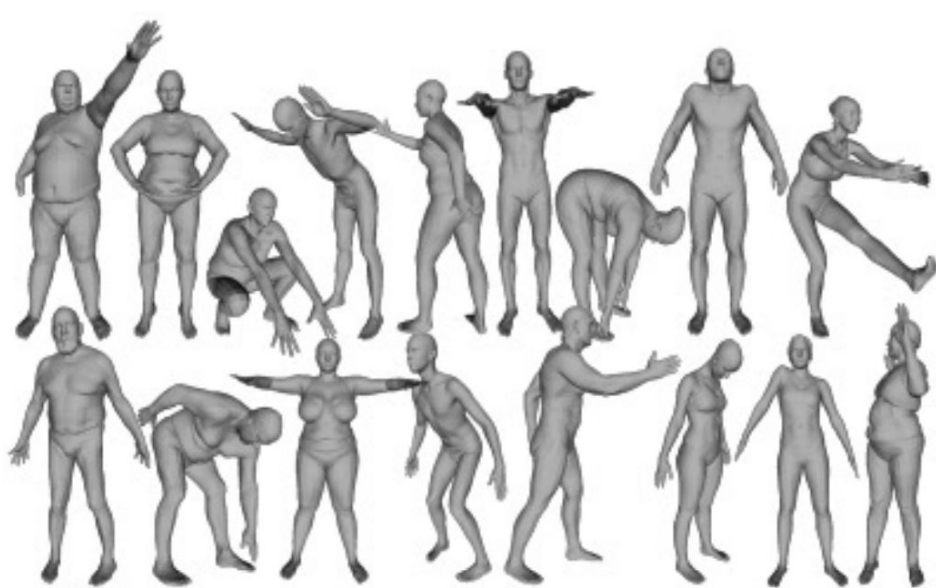
SMPL aligns well
with the image

Side view

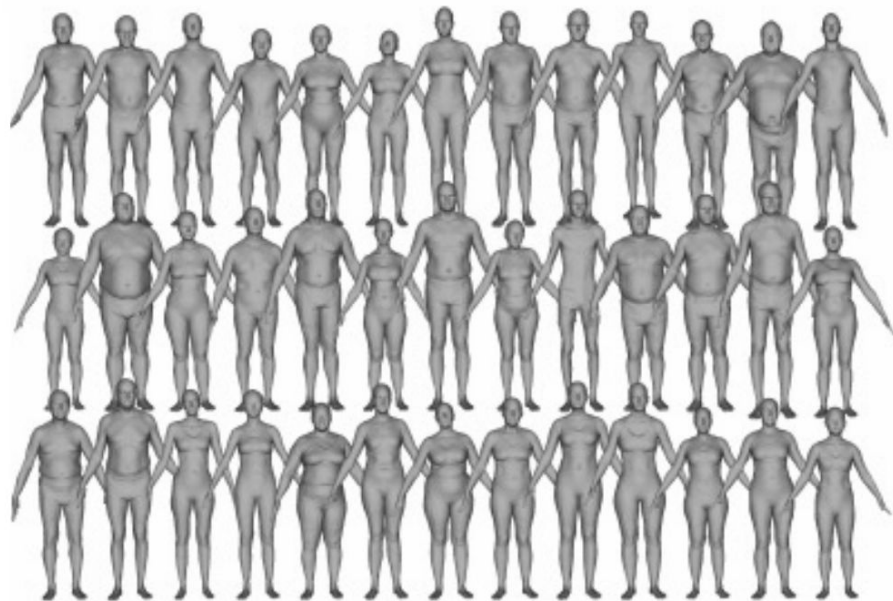


Side view is
incorrect

Solution: Pose and Shape Priors

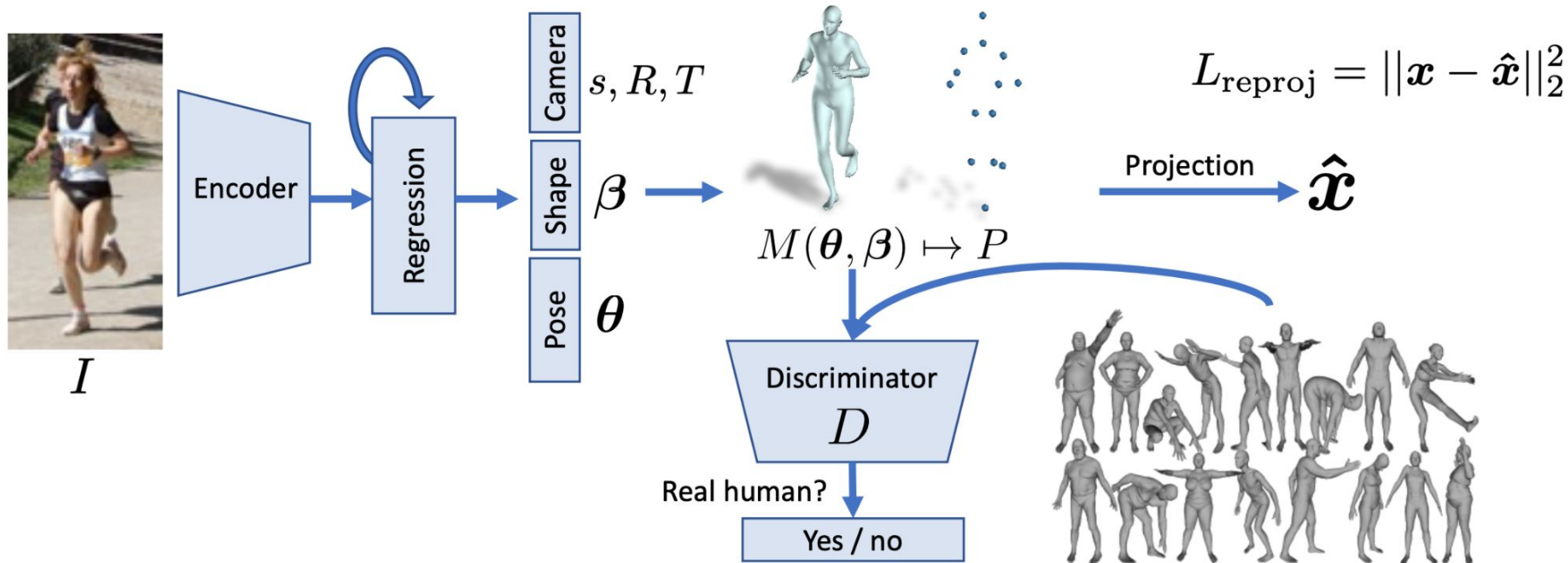


$E_{\theta}(\vec{\theta})$ Pose Prior



$E_{\beta}(\vec{\beta})$ Shape Prior

Regression based approach



Compare optimization and learning based fitting

Optimization (eg. SMPLify)

- ✓ Better **accuracy**, if initialised well.
- ✓ **Feedback loop**

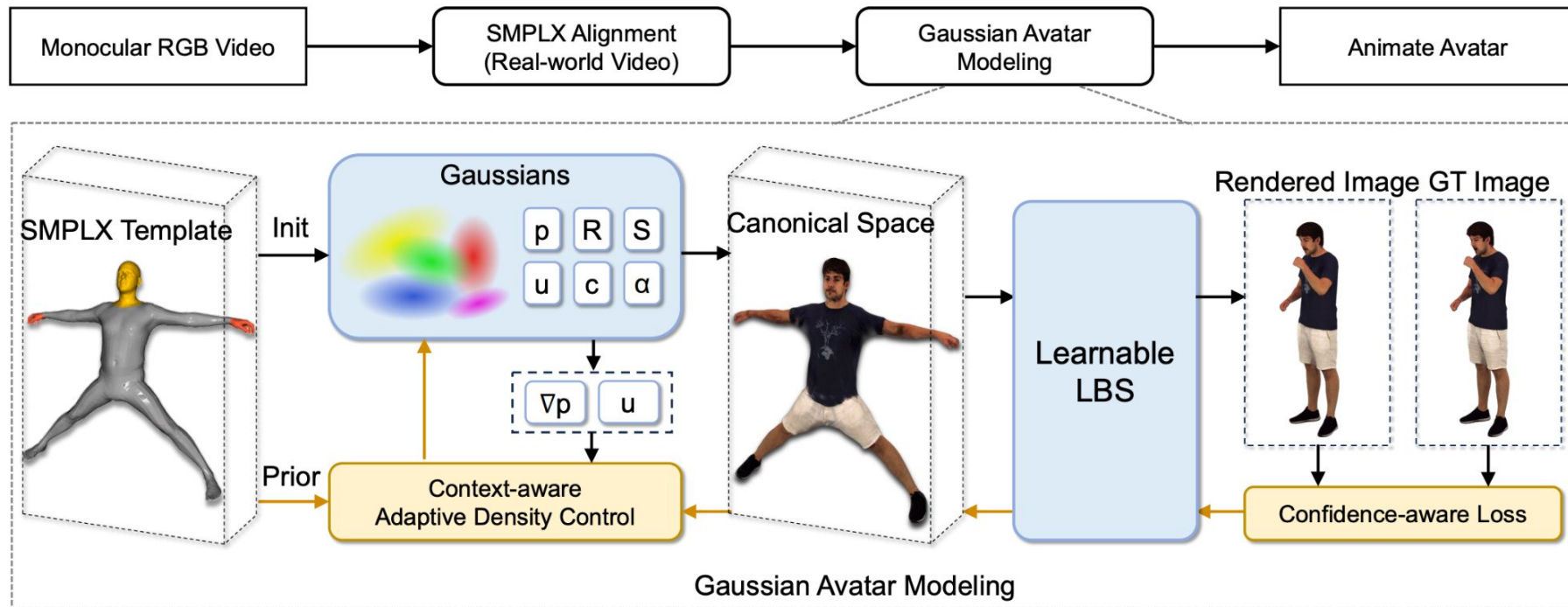
- **Initialization is required**

Learning based (eg. HMR)

- ✓ **Automatic**
- ✓ Leverages data prior

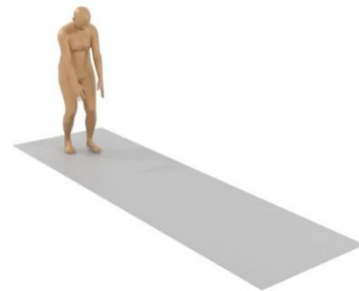
– **Lower accuracy.**
– **No feedback loop**

Application



Application

"A person is skipping rope."



Thank you